Nombre: Ing. Santiago Pila
Proyecto Reto TCS

Solución Proyecto “SmartBancs App”

El presente proyecto detalla la presentación de una solución para el reto planteado para el caso práctico integral “SmartBancs App”, cumpliendo con las diferentes reglas de uso, capacidad, diseño arquitectónico, observabilidad, contexto y restricciones solicitadas.

1. Arquitectura de la solución

La arquitectura de “SmartBancs App” se fundamenta en un modelo de microservicios orientados a eventos (Event-Driven Architecture). Este enfoque distribuye la carga de trabajo de manera asíncrona, permitiendo aislar el flujo transaccional crítico de las tareas secundarias que demandan un mayor tiempo de procesamiento o que interactúan con sistemas legados de menor rendimiento.

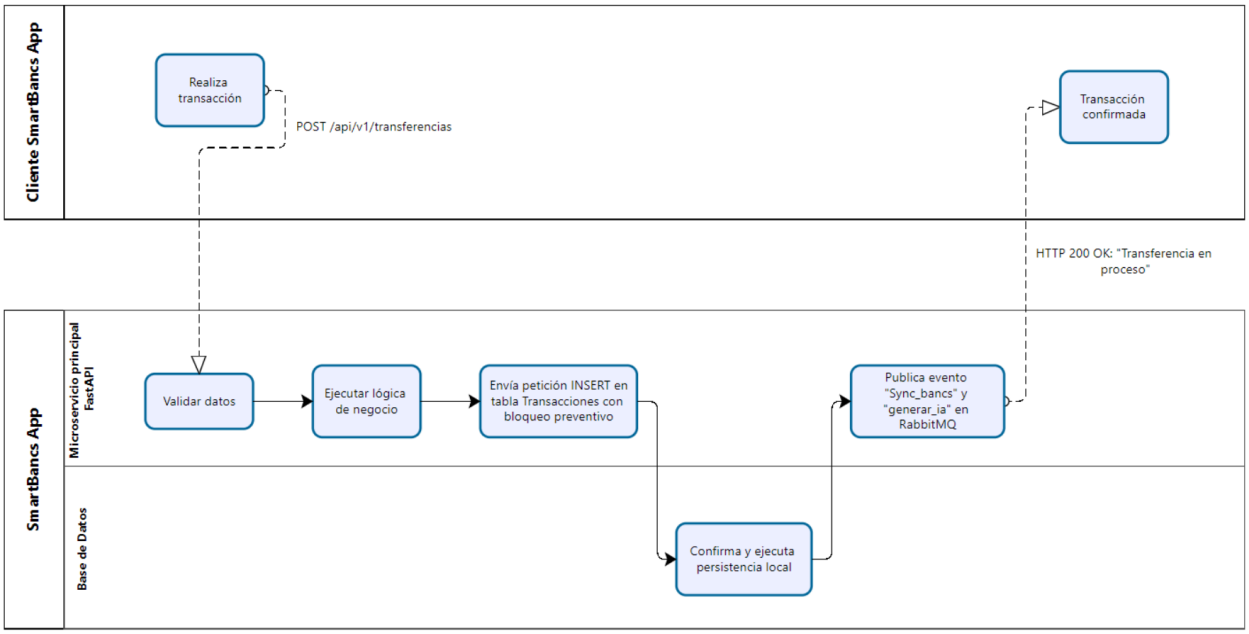


Figura 1. Flujo principal de la solución.

Para poder cumplir con el procesamiento de transacciones solicitadas (10000 transacciones por segundo (TPS)) y garantizar que las transferencias se completen en menos de dos segundos, el sistema está dividido en tres bloques operativos principales.

- Flujo Transaccional Core (Síncrono): Este bloque considera el envío de la petición del cliente a un microservicio principal que actúa exclusivamente como validador y despachador. Este servicio persiste la transacción de forma inmediata en una base de datos relacional y retorna una respuesta de éxito al usuario. El ciclo de vida de la petición HTTP finaliza aquí, lo que asegura una mínima latencia de procesamiento.

- Flujo de Inferencia de IA (Asíncrono): Mientras dura la persistencia de la transacción, el servicio principal emite un evento dirigido a un Message Broker. Un microservicio independiente de inteligencia artificial consume este evento en un segundo plano para

generar recomendaciones financiera, garantizando que este análisis realizado por el modelo no bloquee ni retrase el flujo transaccional principal.

- Flujo de Integración Legada (Asíncrono): Simultáneamente con el anterior evento, se emite un segundo evento destinado a la actualización del sistema central “Bancs”. En este bloque un Worker especializado empezará a consumir estos eventos aplicando técnicas de rate limiting o procesamiento por lotes. Esta técnica genera un amortiguador que protege al sistema heredado de recibir un volumen masivo de consultas directas, lo que evita la degradación de su rendimiento.

2. Decisiones Técnicas

Este reto plantea un escenario tecnológicamente agnóstico, es por este motivo que la selección del stack tecnológico se justifica de forma rigurosa en función del rendimiento, la seguridad, escalabilidad y capacidad de entregar un MVP ejecutable.

Lenguaje y Framework:

Se escogió el lenguaje de programación Python junto al Framework FastAPI debido a que este es un framework moderno basado en el estándar ASGI, que permite un manejo nativo de operaciones asíncronas de I/O. Esta capacidad es crítica para despachar tareas hacia el bloque bróker de mensajería sin necesidad de bloquear el Event Loop, cumpliendo con la restricción de tiempo de respuesta menor a 2 segundo. Además, Python facilita la integración directa de los ecosistemas de análisis de datos y machine learning necesarios para el bloque de transformación y modelo de IA.

Base de Datos Relacional:

Debido a que se requiere garantizar la consistencia transaccional (ACID) frente a una alta concurrencia, PostgreSQL maneja de manera eficiente el control de concurrencia multiversión (MVCC) debido a que permite aplicar bloqueos preventivos a nivel de fila, elemento necesario para mitigar de forma nativa las condiciones de carrera para diferentes operaciones (como la actualización de saldos o el registro de transacciones simultáneas).

Broker de Mensajería:

Para poder dirigir la comunicación asíncrona hacia el servicio de IA y el core legado, RabbitMQ fue la opción escogida debido a que proporciona enrutamiento avanzado, persistencia de mensajes en disco y acuses de recibo (ACKs). Si el sistema “Bancs” sufre una caída por saturación, los mensajes permanecen seguros en la cola hasta que el servicio se restablezca, lo que garantiza que no exista ninguna pérdida de datos financieros.

Infraestructura como código

Para poder cumplir con el requisito obligatorio de entregar un entorno de desarrollo ejecutable con un solo comando, se decidió implementar la contenerización de todos los servicios. Esto asegura que exista paridad entre el entorno de desarrollo y el de producción, lo que facilita el encapsulamiento del microservicio principal. La base de datos, el mock de IA y las herramientas de observabilidad dentro de un ecosistema aislado.

3. *Integración con Bancs*

Para la estrategia de sincronización realizada para la integración del core legado “Bancs” se diseñó un patrón de consistencia eventual para realizar operaciones sin saturar el sistema central. Esta estrategia cuenta con diferentes pasos para asegurar su correcto funcionamiento.

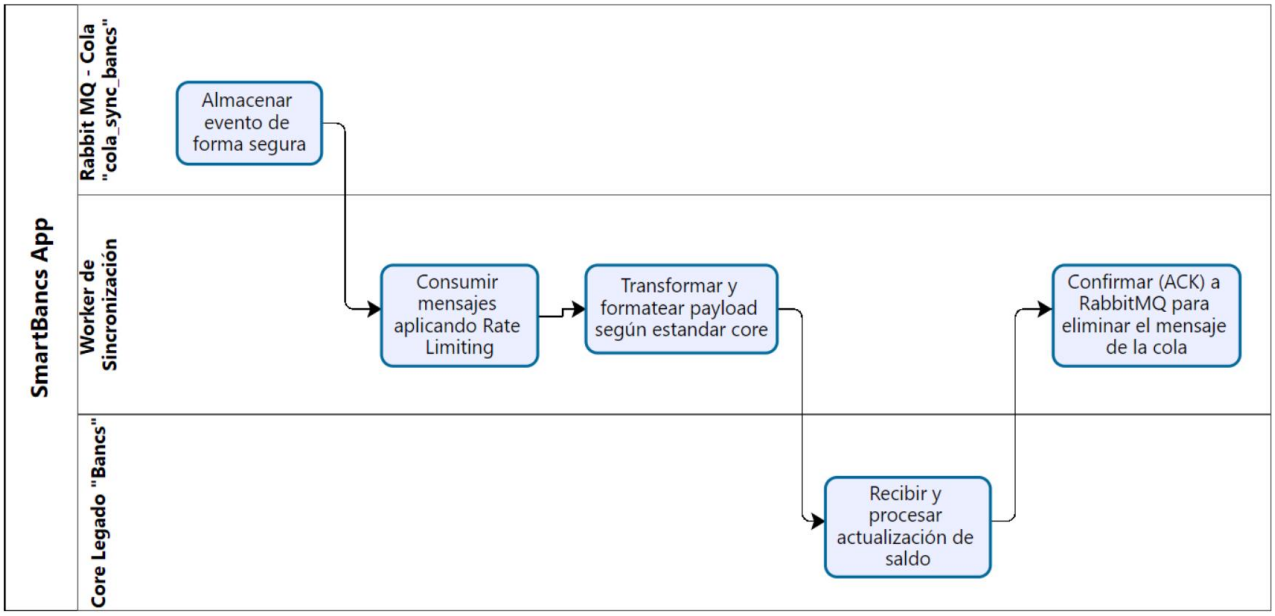


Figura 2. Diagrama de flujo transaccional entre la API y el Core Legado.

1. Encolamiento de Eventos: Cada vez que la base de datos principal PostgreSQL confirma una transacción exitosa, el microservicio publica un mensaje en una cola dedicada de RabbitMQ que contiene “sync_bancs_queue”
2. Control de flujo: Un microservicio Worker independiente consume esta cola aplicando un límite de procesamiento estricto (por ejemplo,100 peticiones por segundo), lo que permite que el Worker actúe como un amortiguador que inyecta las transacciones a Bancs al ritmo máximo que el sistema puede procesar de forma segura sin afectar su rendimiento.
3. Transformación y limpieza de datos: Previo a consolidar grandes volúmenes de datos transaccionales, se ejecuta un proceso ETL que consta de un script programado que toma lotes de datos sin procesar, limpia valores nulos, estandariza formatos de fechas y monedas, para posteriormente estructurarlos en un formato optimizado tanto para Bancs como para los repositorios analíticos de IA.

4. *Manejo del Modelo de IA*

Para garantizar que las recomendaciones financieras no bloqueen el flujo transaccional principal, la inferencia se maneja de forma completamente asíncrona mediante el uso de un servicio desacoplado, gestionando su ciclo de vida en producción siguiendo diferentes políticas.

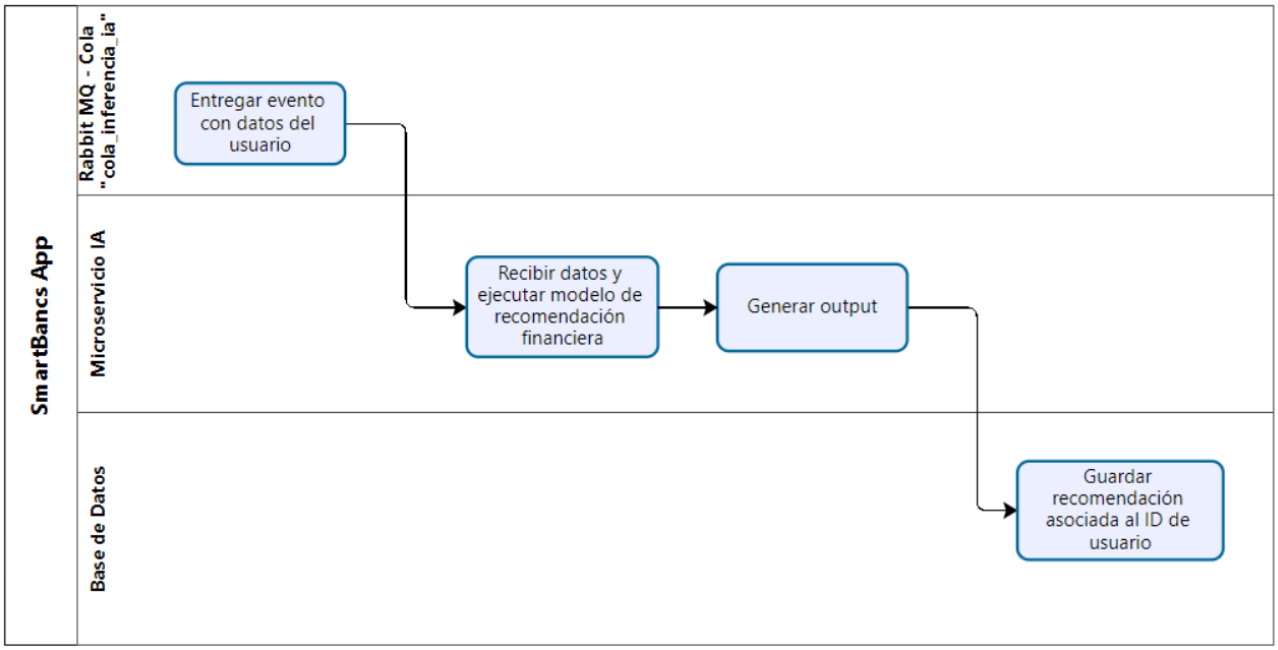


Figura 3. Diagrama de flujo de funcionamiento de microservicio de IA.

- El modelo predictivo requiere actualización constante. El script de transformación ETL procesa el flujo transaccional diario, estructurando un conjunto de datos limpio que se almacena en un Data Lake. Este repositorio estructurado alimenta las tuberías (pipelines) de reentrenamiento del modelo durante ventanas donde exista bajo tráfico.

- En el entorno de producción, se implementan herramientas de observabilidad sobre la distribución estadística de los datos de entrada. Si los patrones de consumo de los usuarios cambian drásticamente (como un aumento atípico de microtransacciones, por ejemplo), el sistema detecta esta desviación de datos y dispara alertas automatizadas para programar un reentrenamiento, asegurando que el modelo mantenga una precisión predictiva.

- La ingesta y evaluación de redes neuronales es intensiva en cómputo. Para evitar la competencia de recursos en el backend transaccional, el modelo de IA se desplegó sobre clústeres aislados (contenedores independientes) con asignaciones rígidas y cuotas limitadas de uso de CPU y memoria, garantizando de esta manera que un pico de solicitudes de predicción no afecte la disponibilidad del microservicio principal.

5. *Respuesta y Gestión de Incidentes Críticos*

Para analizar la respuesta y la gestión a diferentes casos críticos de incidentes, se realizaron las pruebas dentro del proyecto para evaluar sus resultados:

Caso de Prueba

Durante un pico transaccional de quincena, se presenta un incremento severo en latencia, errores de timeout en la base de datos y bloqueos mutuos (deadlocks).

Acciones Inmediatas de Contención:

Para poder estabilizar el sistema de manera rápida y restablecer el servicio a los usuarios, se aplicaron las siguientes medidas de mitigación de emergencia:

1. Resolución de Bloqueos (Deadlocks): Mediante scripts de automatización, se identifican los Process ID (PID) mediante los logs de monitoreo y se finalizan abruptamente las conexiones bloqueadas para liberar tablas transaccionales.
2. Balanceo y Escalado de Cargas: Se aumenta dinámicamente el número de réplicas del microservicio principal (Horizontal Pod Autoscaling) para distribuir el tráfico masivo de quincena entre más nodos y balancear las cargas adecuadamente
3. Ajuste de configuraciones de Red: Se reducen temporalmente los timeouts en el API Gateway y en el Connection Pooler para evitar que las peticiones encoladas sigan consumiendo hilos de ejecución en el servidor mientras se recupera la base de datos.

Escalamiento, Post-Mortem y Prevención

Una vez que se ha estabilizado la plataforma, el incidente se documenta utilizando el marco de reporte de incidentes (Post-mortem) que se estructura de la siguiente manera:

- Contexto e impacto: Número de usuarios afectados, duración del incidente y tasa de transacciones fallidas
- Timeline: Registro cronológico de la detección a través de las métricas de observabilidad hasta la mitigación
- Causa Raíz: Análisis profundo que identifique la consulta SQL no optimizada o la falla de configuración que detonó el cuello de botella inicial.

Además, se definieron algunas acciones preventivas tanto en infraestructura como en código. A nivel de código se realizó la refactorización de la estructura de las transacciones SQL, asegurando el uso correcto de índices en las tablas de Usuarios y Transacciones para agilizar las operaciones de lectura/escritura y minimizar el tiempo de retención de los bloqueos a nivel de fila.

A nivel de infraestructura se propuso implementar un Connection Pooler robusto (como PgBouncer) configurado específicamente para gestionar miles de conexiones transitorias sin sobrepasar el límite máximo de conexiones activas tolerado por la base de datos. Además se tomó en cuenta la integración de alertas tempranas en la telemetría antes de alcanzar el 80% de límite de conexiones.

6. Diseño de Observabilidad

Para garantizar la salud de la solución 'SmartBancs App' y facilitar el diagnóstico rápido de incidentes, el diseño de observabilidad se estructura sobre la centralización de logs y la trazabilidad distribuida.

- **Trace ID Único:** Se genera un UUID en la capa del API Gateway/Microservicio que viaja como metadato a través de RabbitMQ hacia el servicio de IA y el worker de Bancs. Esto debido a que permite reconstruir el ciclo de vida completo de una

transacción fallida a través de múltiples microservicios desacoplados, reduciendo el tiempo de resolución (MTTR).

- **Latencia de Consultas SQL (con métricas de bloqueo):** Se instrumentó el driver de base de datos para medir en milisegundos cuánto tarda la instrucción SELECT FOR UPDATE. Un pico en esta métrica es el indicador temprano para identificar la degradación del servicio o predecir un posible deadlock en quincena antes de que afecte masivamente a los usuarios.
- **Tasa de Encolamiento vs. Consumo en RabbitMQ:** Se monitorea la profundidad de las colas (cola_sync_bancs y cola_inferencia_ia). Si los mensajes se acumulan más rápido de lo que se procesan, indica que el modelo de IA está sufriendo inanición de recursos o que el sistema legado Bancs no está respondiendo a la tasa de rate limiting configurada.